

RERA Operation Manual

Overview

RERA is a containerized validation infrastructure tool. It does not replace your proprietary physics simulator; instead, it acts as a **Bayesian Orchestrator**. It intelligently commands your simulator to find the exact mathematical boundaries where your autonomous vehicle (AV) fails, isolates the root cause, and generates ISO 21448 (SOTIF) compliance artifacts.

This manual outlines the 4-step process for integrating your proprietary data and executing the certification pipeline.

Step 1: Configuration (default.yaml)

Before running the engine, you must define your vehicle's Operational Design Domain (ODD) bounds. You do not need to edit any Python code; all execution parameters are controlled via the default.yaml file.

1. Open default.yaml.
 2. Set your **Operational Bounds**: Adjust the minimum and maximum ranges for the environments you want to test (e.g., traffic_density: [0.0, 1.0]).
 3. Set your **Hardware Utilization**: Adjust target_vram_utilization (default 0.90). RERA will automatically probe your available CUDA memory and scale its parallel tensor batching to maximize your specific GPU without causing Out-of-Memory (OOM) crashes.
 4. Set your **Simulation API Hook**: Point the map_path to your proprietary USD scene or map file.
-

Step 2: Ingesting Your Fleet Data (The "Anchor" Seeds)

By default, RERA explores the parameter space randomly. To make the test highly specific to your vehicle, you must feed it your real-world fleet disengagement logs (e.g., ROS2 .bag or .mcap files).

1. Query your data lake for specific disengagement events.
2. Extract the environmental data at the time of the human takeover (e.g., *Weather: Rain, Traffic: Heavy, Speed: 45mph*).

3. Format these values into RERA's input tensor schema and pass them to the FleetDataIngestor module.
 4. **Result:** RERA will now use your real-world crashes as "Anchor Seeds," mathematically exploring the geometry of the failure outward from that exact point.
-

Step 3: Execution

RERA is fully containerized to prevent dependency conflicts with your internal tech stack.

1. Build the Engine Run this from the root directory to compile the CUDA/PyTorch environment:

Bash

```
docker build -t RERA -f docker/Dockerfile .
```

2. Launch the Certification Run Mount your local results directory so the output artifacts persist after the container spins down:

Bash

```
docker run --rm --gpus all -v "${PWD}/results:/app/results" RERA
```

Note: The engine will output timestamped, enterprise-grade logs to the console and save a persistent .log file in the results directory for your MLOps tracking tools (e.g., Datadog).

Step 4: Reading the Certification Artifacts

Upon completion, navigate to your /results folder. RERA will have generated a suite of audit-grade artifacts for your safety and engineering teams:

1. The SOTIF Compliance Dossier (sotif_compliance_dossier.json)

- This JSON file states the system's verified Safe Operating Envelope, the number of edge cases discovered, and the primary systemic fault hazard.

2. The Audit Dashboard (certification_report.html)

- Open the interactive HTML file in any browser. Review the **Robustness Margin (RM)** to see the statistical health of your stack. Use the "Regime Clusters" to understand if failures are grouped around specific conditions (e.g., "Dense Traffic + Low Friction").

3. The Causal Root-Cause & Auto-Repair Patch

- **Who it's for:** Control Systems & ML Engineers.
 - **What to do with it:** Review the terminal output or log file to view the Causal Directed Acyclic Graph (DAG). It will tell you exactly *why* the vehicle failed (e.g., Sensor Noise -> Estimator Drift -> Control Lock). Apply the suggested mathematical policy repair (e.g., *Adjust PID gain to -0.09*) to your codebase to instantly patch the vulnerability.
-

Step 5: Continuous Integration (CI/CD) Setup

RERA can be run automatically as a safety tool.

1. Add the provided `.github/workflows/RERA-certify.yml` file to your repository.
2. Whenever an engineer submits a Pull Request that modifies the AV's perception or control stack, GitHub Actions will trigger RERA.
3. RERA will compare the new software's "Safe Operating Envelope" against the baseline.
4. If the new code introduces a statistical regression (shrinking the safe volume), the pipeline will automatically block the deployment and flag the PR.